

MODULE 6 · SCHEDULING, RESOURCES & SCALING

DaemonSets

One Pod on every matching node — no replica count needed

Some workloads must run everywhere, not just somewhere

A Deployment gives you **N Pods, placed wherever the scheduler finds room** — great for stateless apps, useless for a log shipper or metrics agent that needs to sit on **every single node**.

A **DaemonSet** flips the model: instead of a count, you declare a Pod template and Kubernetes places exactly one copy on each node that matches — no matter how many nodes that turns out to be.

BY THE END YOU CAN

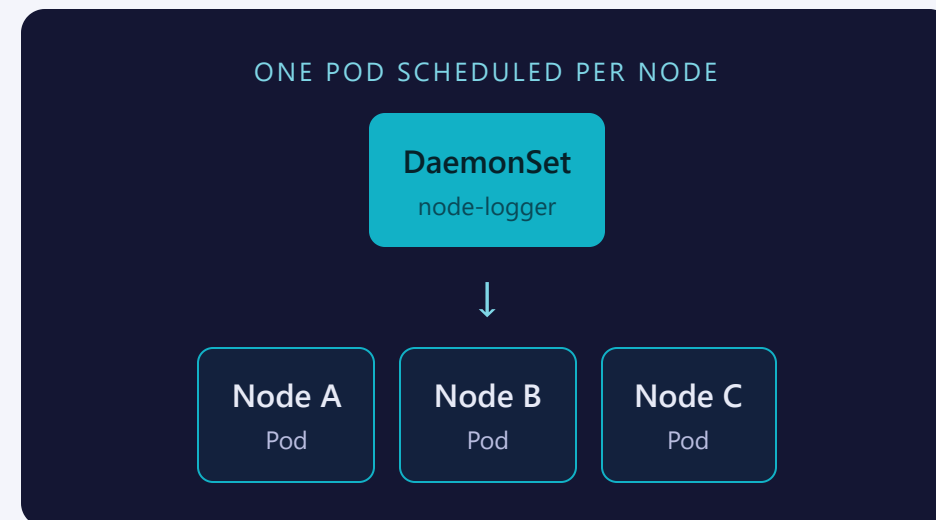
- Explain the one-pod-per-node model
- Add a control-plane toleration
- Say why DaemonSets have no replicas
- Read DESIRED/CURRENT/READY correctly

CONCEPT

The node set decides the Pod count, not you

- **Every matching node** gets exactly one copy of the Pod template
- **Node joins the cluster** → the DaemonSet controller schedules a Pod there automatically
- **Node leaves the cluster** → its Pod is garbage collected with it

A nodeSelector or affinity on the template can narrow "every node" down to a subset — but there's still no number to set.



Per-node agents, not per-app replicas

AGENT TYPE	WHY IT NEEDS EVERY NODE
Log collector	Ships each node's container logs — miss a node, lose its logs
Metrics agent	Scrapes host-level CPU/memory/disk stats no other node can see
Network plugin	Wires up Pod networking locally — must exist before other Pods can run
Storage plugin	Mounts volumes for Pods scheduled on that specific node

MENTAL MODEL

These agents aren't "the app" — they're infrastructure that has to be co-located with every node's workloads, one instance each.

A Pod template — and a toleration, not a count

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-logger
  namespace: training
  labels:
    app: node-logger
spec:
  selector:
    matchLabels:
      app: node-logger      # must match template labels
  template:
    metadata:
      labels:
        app: node-logger
    spec:
      containers:
        - name: logger
          image: busybox:1.36
          command:
            - /bin/sh
            - -c
            - "while true; do echo \"$(date) - Running on node: $(hostname)\"; sleep 5; done"
          resources:
            requests:
              cpu: 10m
              memory: 16Mi
            limits:
              cpu: 50m
              memory: 32Mi
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule      # land here too
```

selector / template — same shape as a Deployment's, no replicas field · **tolerations** — without this, the control-plane's NoSchedule taint keeps this Pod off it entirely.

There's no imperative shortcut for this one

OBJECT	IMPERATIVE GENERATOR
Deployment	<code>kubectl create deployment</code>
Job	<code>kubectl create job</code>
DaemonSet	— none —

THE BRIDGE TRICK, ADAPTED

No generator to bootstrap from — you write the YAML by hand. Validate it against the API server before it does anything real:

```
$ kubectl apply --dry-run=server -f daemonset.yaml # checked, not applied
```

SEE IT

Node count changes, Pod count follows

```
$ kubectl scale daemonset node-logger --replicas=5 -n training
Error from server (NotFound): ...
# no /scale subresource – always fails

$ kubectl get daemonset node-logger -n training
NAME           DESIRED   CURRENT   READY
node-logger     2         2         2
# 2 == node count, not a chosen number
```

Older kubectl versions print a friendlier daemonsets do not support scaling — same root cause either way.

SIZING FOLLOWS THE NODE SET

+ node joins

Pod scheduled there

– node removed

its Pod is gone with it

Where people trip

- **kubectl scale on a DaemonSet.** No `/scale` subresource exists — it fails on every version, just with different wording.
- **Forgetting the control-plane toleration.** The Pod doesn't error — it just never lands there, silently.
- **Reading only DESIRED.** That number isn't one you set — it's whatever the current matching node count is.
- **A nodeSelector that's too narrow.** Silently excludes nodes you expected to get a copy — check node labels first.

Commands to keep at hand

```
$ kubectl apply -f daemonset.yaml           # create or update
$ kubectl rollout status daemonset/NAME -n NS --timeout=60s # wait until ready
$ kubectl get daemonset NAME -n NS          # DESIRED / CURRENT / READY
$ kubectl get pods -n NS -l app=LABEL -o wide # one row per node, see NODE col
$ kubectl logs -n NS -l app=LABEL --tail=5   # per-node output
$ kubectl delete -f daemonset.yaml           # removes every Pod too
```

Tip: `-o wide` is how you prove "one per node" — check the `NODE` column, not just the count.

RECAP

DaemonSets in one breath

- One Pod per matching node — the node set sets the count, not you
- Node joins → Pod appears; node leaves → Pod is gone with it
- No replicas field, no /scale subresource — `kubectl scale` always fails
- Tolerations let the Pod also land on tainted nodes, like the control-plane

HANDS-ON

Now run Lab 6.5
[labs/6.5/](#)

Next: [6.6 — Jobs & CronJobs](#)